TheoInf 1
Woche 11

Nachtrag zu

Goto

▸ Neue Sprachfeatures

▸ 1. break: Abbruch einer Schleife

```
loop x
   ⟨PTeil 1⟩
   break
   ⟨PTeil 2⟩
end
⟨P⟩
```

```
      loop x
         ⟨PTeil 1⟩
         goto L
         ⟨PTeil 2⟩
      end
L:    ⟨P⟩
```

Dasselbe geht auch für alle anderen Schleifen (also *while* und *for*)

Dabei müssen ⟨PTeil 1⟩ und ⟨PTeil 2⟩ keine eigenständigen Programme sein, sondern es darf z.B. in ⟨PTeil 1⟩ ein *if* stehen, dessen *end* in ⟨PTeil 2⟩ kommt.

Beispiel: Durchsuchen einer Liste a nach einem Wert x

```
Trefferposition ← 0
for i ← 0, …, count(a)−1
   if a_i = x
      Trefferposition ← i
      break
   end
end
```

Hinweis: *break* wird immer so realisiert, dass es die innerste Schleife abbricht, in der es steht

Beispiel:

```
z ← 0
for i ← 2, …, n
   for j ← 1, …., n
      if j ≥ i
         break
      end
      z ← z + i · j
   end
end
```

ist dasselbe wie

```
z ← 0
for i ← 2, …, n
   for j ← 1, …., i−1
      z ← z + i · j
   end
end
```

▸ **2. continue:** bricht aktuellen Schleifendurchlauf ab und fährt ggF mit dem nächsten fort.

```
loop x
   ⟨PTeil 1⟩
   continue
   ⟨PTeil 2⟩
end
⟨P⟩
```

```
      loop x
         ⟨PTeil 1⟩
         goto L
         ⟨PTeil 2⟩
L:    end
      ⟨P⟩
```

Beispiel: Summe aller nicht durch 5 teilbaren Zahlen ≤ n:

```
z ← 0
for i ← 1, …, n
   if i % 5 = 0
      continue
   z ← z + i
end
```

berechnet dasselbe wie:

```
z ← 0
for i ← 1, …, n
   if i % 5 ≠ 0
      z ← z + i
   end
end
```

▸ **Konvention:** Ab sofort dürfen Sie *break* und *continue* auch in Extended-While und Extended-Loop-Programmen benutzen. 🙂

Paradoxien

»Dieser Satz ist falsch«

»Derjenige, der genau die Leute rasiert, die sich nicht selbst rasieren«

»Die Menge aller Mengen, die sich nicht selbst enthalten« (Russelsche Antinomie)

»Die kleinste natürliche Zahl, die nicht eindeutig in ≤ 100 Zeichen definiert werden kann« ⇦ das sind 86 Zeichen

Berechenbarkeit

▸ **Beobachtung:** Viele Berechnungsmodelle/Programmiersprache lassen sich wechselseitig ineinander übersetzten, z.B.:

- *while*-Programme
- *goto*-Programme
- Turingmaschine
- Java
- Pyhton
- …

⇒ Diese Berechnungsmodelle sind *Turing-mächtig*, *Turing-vollständig*

▸ **Church-Turing-These:**
»Berechenbar ist genau das, was man mit einem Turing-mächtigen Berechnungsmodell berechnen kann.«

▸ **Hypercomputation**
Berechnung, die über die Grenzen des Turing-berechnbaren hinaus geht.

Halteproblem

Gegeben ein Programm p und eine Eingabe s, entscheide, ob b gestartet auf s anhält oder ob es sich "aufhängt".

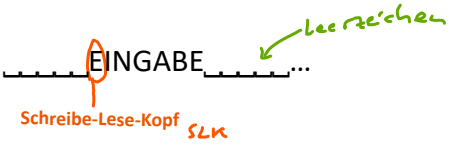
Beobachtung: Das Halteproblem ist für jedes Turing-mächtige Berechnungsmodell nicht Turing-berechenbar.

Turingmaschine

Die Turingmaschine ist ein aufgemotzter endlicher Automat, der

1. die Eingabe nicht nur lesen sondern auch verändern kann,
2. die Eingage nicht nur vorwärts sondern auch rückwärts durchlaufen kann,
3. links und rechts neben der Eingabe beliebig viel Platz für weitere Zeichen hat.

Am Anfang ist die Eingabe so etwas : …. ␣␣␣␣␣EINGABE␣␣␣␣␣…



In jeder Runde liest die TM mit dem SLK ("Schreibe-Lese-Kopf") ein Zeichen vom Band ein, überschreibt dann dieses Zeichen und bewegt anschließend den SLK um eine Position nach links oder rechts.

- Σ: endliches Alphabet
- Q: endliche Zustandsmenge
- s ∈ Q: Startzustand
- F ⊆ Q: akzeptierenden Endzuständen
- Γ: endliches Bandalphabet, Σ ⊂ Γ
- ␣ ∈ Γ \ Σ: spezielles Leerzeichen
- δ: Übergangsfunktion

▸ Startsituation

Die Maschine befindet sich im Zustand s.
Das gesamte Band ist mit Leerzeichen ␣ gefüllt, außer dem Teil, der die Eingabe enthält.
Der SKL steht auf dem ersten Zeichen der Eingabe.

▸ Schluss

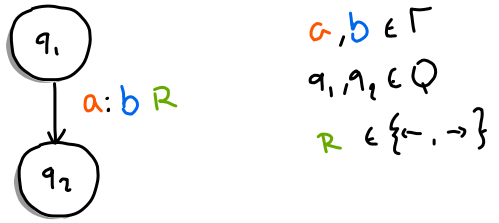
Sobald die Maschine einen Zustand erreicht, aus dem heraus sie nicht mehr mit einem legalen Übergang heraus kommt, hält sie an.
Der der Eingabestring wird akzeptiert, wenn die Maschine in einem Zustand ∈ F anhält.

▸ Übergangsfunktion

δ: {Q \ F} x Γ → Q x Γ + {"←", "→"}

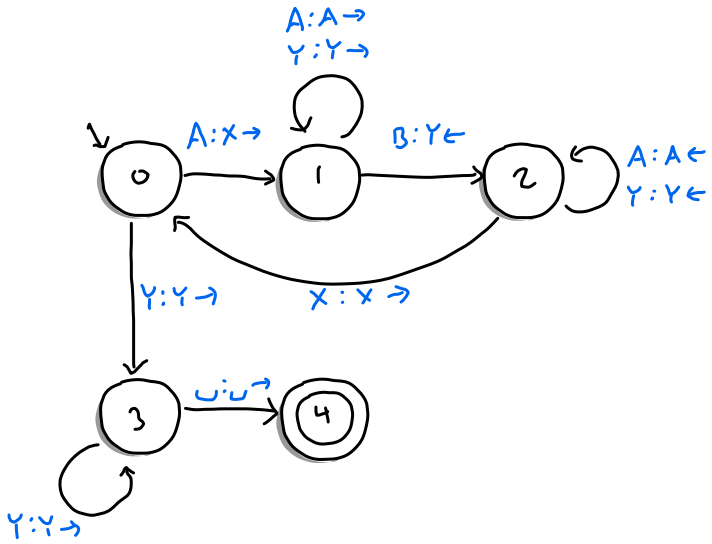
"←" bedeutet, dass der SLK eine Position nach links gehen soll
"→" bedeutet, dass der SLK eine Position nach rechts gehen soll

Wenn $\delta(q_1, a) = (q_2, b, R)$ bedeutet das:
»Wenn sich die Maschine im Zustand q_1 befindet und vom Band mit dem SLK das Zeichen a einliest, dann überschreibt sie das Zeichen mit b, bewegt anschließend den SLK in die Richtung R und geht in den Zustand q_2 über.«



▸ **Beispiel**

$\Sigma = \{A, B\}$
$Q = \{0, 1, 2, 3, 4\}$
$s = 0$
$F = \{4\}$
$\Gamma = \{A, B, \sqcup, X, Y\}'$



diese Turingmaschine akzeptiert die Sprache $L = \{A^n B^n \mid n \in \mathbb{N}\}$